

mimics the functionality of a pointer that points to an object within in your nib (xib) file. Then, of course, are the actions which we had elaborated on earlier.

```
-(IBAction) bnClicked:(id)sender;
```

IBAction is a special reserved keyword. This also implies a return type of void for your method named **bnClicked** (this can be anything you want).

The (id)sender is the argument which tells your code which was the origin for the stimulus (a tap, flick etc) for your action to be executed, if you want it to be executed irrespective of who triggers it you can also just declare it as:

```
-(IBAction) bnClicked;
```

One important point of note here is that in Objective-C 2.0, the following statements are identical, this will bring in some familiarity to those who are used to Java/C++ “.” Notation.

```
myVar = [arbitObject property];
myVar = arbitObject.property
similarly
arbitObject.property = myVar;
arbitObject setProperty:myVar];
```

3.1 Creating an outlet

To create an outlet, select the “File’s Owner” item in your digit.xib file and open its Identity Inspector. Below, where you created the **bnClicked:** action, there is an option to create a Class Outlet, by double clicking the + button. Add an outlet called **nmTextField** and the Type should be set to **UITextField**.

To differentiate further between actions and outlets, an action is a method to handle events invoked by the views (till now we have been dealing with only single view applications) while an outlet allows your code to programmatically modify or reference a view on the View window.

Now again select the **File’s Owner** and choose **File > Write Class File** item from the menu. This is done to auto generate



Typical events for a text field